

Tech Challenge – IA Aplicada à Saúde – Fase 2

Otimização de Random Forest com Algoritmo Genético, Explicabilidade
com LLM e Disponibilização por API

Marcelo Viana de Araujo – RM372707

Nirton Afonso de Oliveira Filho – RM370221

Valkiria Nonato de Faria – RM373256

Benicio Antonio Cardoso – RM371527

Pós-Graduação em IA para Devs

2026

Repositório GitHub da Fase 2:

<https://github.com/Val-Faria/tech-challenge-ia-saude/tree/Nirton>

Repositório da Fase 1:

[https://github.com/mvaraujo1977/TechCahllenge_Tireoide/](https://github.com/mvaraujo1977/TechChallenge_Tireoide/)

API pública implantada no Render:

<https://tech-challenge-ia-saude.onrender.com>

Vídeo de demonstração:

<https://www.youtube.com/watch?v=IXaI20e1feE>

Sumário

1	Resumo	4
2	Introdução	4
2.1	Escopo do projeto	5
3	Objetivos	5
3.1	Objetivo geral	5
3.2	Objetivos específicos	5
4	Arquitetura e Organização da Solução	6
5	Base de Dados e Preparação	7
6	Modelo de Referência	8
7	Otimização com Algoritmo Genético	8
7.1	Representação dos genes	8
7.2	Operadores evolutivos	8
7.3	Função de aptidão	9
7.4	Protocolo experimental	9
8	Resultados e Comparação dos Modelos	11
8.1	Importância global das variáveis	12
9	Integração com Modelo de Linguagem	13
9.1	Engenharia de prompt	14
9.2	Avaliação da explicação	14
10	API, Monitoramento e Autoscaling	15
10.1	Implantação pública no Render	15
10.2	Configuração de autoscaling	15
11	Testes e Reprodutibilidade	16
12	Desafios Enfrentados e Soluções Implementadas	16
12.1	Pré-processamento durante a validação cruzada	16
12.2	Desbalanceamento da base e escolha do recall	17
12.3	Tempo de execução dos experimentos	17
12.4	Execução no Colab e disponibilização da API	17
12.5	Versão do scikit-learn e carregamento do Joblib	17
12.6	Inclusão do modelo no repositório	18

12.7 Limites da interpretação gerada pela LLM	18
13 Limitações e Considerações Éticas	18
14 Conclusão	18
15 Referências	19

1 Resumo

Este relatório apresenta a evolução do projeto de Inteligência Artificial aplicada à saúde desenvolvido na Fase 1, cujo objetivo é classificar registros clínicos relacionados à tireoide. Na Fase 2, o trabalho concentrou-se na otimização do modelo Random Forest por meio de Algoritmo Genético, na geração de explicações em linguagem natural com um modelo de linguagem de grande porte e na disponibilização do modelo por uma API REST observável.

O Algoritmo Genético representa cinco hiperparâmetros do classificador como genes: número de árvores, profundidade máxima, quantidade mínima de amostras para divisão, quantidade mínima de amostras por folha e estratégia de seleção de atributos. A função de aptidão utiliza, em ordem lexicográfica, o recall, o F1-score e a AUC-ROC médios obtidos por validação cruzada estratificada, mantendo o recall como primeiro critério de seleção. Foram executados três experimentos independentes, com diferentes tamanhos de população, números de gerações e sementes aleatórias.

O melhor experimento alcançou recall médio de 0,9697 na validação cruzada. No conjunto de teste, o modelo otimizado obteve accuracy de 0,9892, precision de 0,8333, recall de 0,9524, F1-score de 0,8889 e AUC-ROC de 0,9947. O modelo original apresentou recall de 0,9048. Portanto, a otimização aumentou a sensibilidade para a classe de hipotireoidismo, embora tenha produzido redução de precision e F1-score.

A integração com a LLM foi estruturada para converter resultados numéricos em uma explicação objetiva, sem apresentar diagnóstico ou recomendação de tratamento. A solução também inclui API em FastAPI, documentação OpenAPI, logs estruturados, métricas Prometheus, testes automatizados e imagem Docker. A API foi efetivamente implantada no Render e validada com uma observação real do conjunto de teste. O projeto mantém ainda uma configuração declarativa de autoscaling para Google Cloud Run como referência de arquitetura. O modelo treinado foi versionado no repositório no formato Joblib. O sistema possui finalidade exclusivamente acadêmica e não substitui avaliação médica.

2 Introdução

Instituições de saúde produzem grandes volumes de dados laboratoriais e clínicos. Modelos de aprendizado de máquina podem auxiliar a identificação de padrões nesses registros, mas aplicações relacionadas à saúde exigem critérios de avaliação compatíveis com o impacto de seus erros. Em uma tarefa de triagem de hipotireoidismo, deixar de identificar um caso positivo pode ser mais prejudicial do que encaminhar um caso negativo para uma análise complementar.

A Fase 1 estabeleceu a base de dados, o processo de limpeza, a análise exploratória e

a comparação inicial de classificadores. O Random Forest foi selecionado como modelo de referência. A Fase 2 dá continuidade a esse trabalho, investigando se uma busca evolutiva consegue encontrar uma combinação de hiperparâmetros mais adequada ao objetivo de ampliar o recall da classe positiva.

Além da otimização, o projeto incorpora uma camada de explicação em linguagem natural. Essa etapa busca tornar os resultados mais compreensíveis, preservando limites claros: probabilidades do modelo não representam risco clínico calibrado, importância global de variável não demonstra causalidade e a saída gerada pela LLM não constitui diagnóstico.

2.1 Escopo do projeto

O escopo da Fase 2 compreende:

- otimização de hiperparâmetros do Random Forest com Algoritmo Genético;
- execução e comparação de três experimentos evolutivos;
- comparação do modelo otimizado com o modelo original da Fase 1;
- interpretação dos resultados e das variáveis globalmente mais relevantes;
- geração de explicações em linguagem natural com uma LLM pré-treinada;
- avaliação estruturada da qualidade da explicação gerada;
- disponibilização do modelo treinado por API REST;
- implementação de testes, logs, métricas e configuração de autoscaling.

3 Objetivos

3.1 Objetivo geral

Otimizar o modelo de classificação de hipotireoidismo desenvolvido na Fase 1 e disponibilizar seus resultados de maneira reproduzível, interpretável e tecnicamente observável.

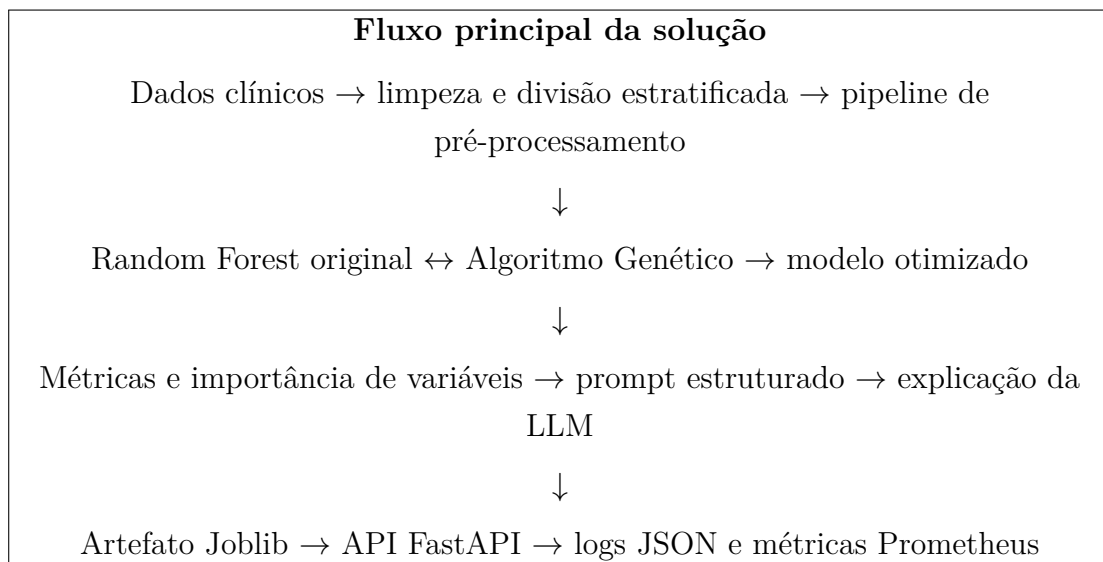
3.2 Objetivos específicos

- representar hiperparâmetros do Random Forest como cromossomos de um Algoritmo Genético;
- implementar operadores de seleção, cruzamento, mutação e elitismo;

- definir uma função de aptidão baseada em métricas relevantes para a classe minoritária;
- conduzir ao menos três experimentos com configurações distintas;
- avaliar o desempenho em dados não utilizados durante a busca;
- comparar quantitativamente os modelos original e otimizado;
- utilizar engenharia de prompt para produzir explicações com linguagem neutra;
- verificar clareza, fidelidade numérica e segurança da explicação;
- fornecer endpoints para saúde do serviço, predição e monitoramento;
- documentar decisões, limitações e possibilidades de implantação.

4 Arquitetura e Organização da Solução

A solução foi organizada de modo que treinamento, inferência e operação permaneçam separados. O notebook concentra a análise, a otimização, a avaliação e a geração do artefato treinado. A API carrega esse artefato e expõe apenas as operações necessárias para inferência e monitoramento.



Os principais componentes versionados são:

- `02_otimizacao_hiperparametros_ag.ipynb`: experimento completo e demonstração no Google Colab;
- `api/main.py`: aplicação FastAPI responsável pela inferência;

- `models/random_forest_otimizado.joblib`: pipeline treinado e pronto para carregamento;
- `tests/test_api.py`: testes automatizados dos endpoints;
- `Dockerfile`: definição da imagem da aplicação;
- `cloudrun/service.yaml`: configuração declarativa do serviço e do autoscaling;
- `requirements.txt`: ambiente completo utilizado no notebook e nos testes;
- `requirements-api.txt`: dependências necessárias para executar somente a API.

A implantação utilizada na demonstração está disponível publicamente em <https://tech-challenge-ia-saude.onrender.com>. O notebook permite selecionar essa API remota ou iniciar a mesma aplicação localmente no ambiente do Google Colab.

5 Base de Dados e Preparação

Foi mantida a base de dados *Thyroid Disease*, disponibilizada no UCI Machine Learning Repository e utilizada na Fase 1:

<https://archive.ics.uci.edu/dataset/102/thyroid+disease>

O problema foi formulado como classificação binária entre registros negativos e registros associados a hipotireoidismo. A base original possui 3.163 registros e 26 colunas, considerando a variável-alvo. Durante a limpeza foram removidos 77 registros duplicados; as etapas de modelagem utilizaram os 3.086 registros restantes, dos quais 2.945 pertencem à classe negativa e 141 à classe de hipotireoidismo.

Os atributos incluem informações demográficas, indicadores clínicos e medições laboratoriais, como idade, sexo, TSH, T3, TT4, T4U, FTI e TBG. Também há indicadores categóricos que informam uso de medicação, cirurgia da tireoide, gestação, presença de outras condições e disponibilidade de cada exame.

A preparação dos dados foi encapsulada em pipelines do `scikit-learn`. Para variáveis numéricas, foi aplicada imputação. Para variáveis categóricas, foram utilizados imputação e One-Hot Encoding. Esse encapsulamento garante que os transformadores sejam ajustados apenas com os dados de treinamento em cada partição, reduzindo o risco de vazamento de informação.

A divisão foi estratificada em treino, validação e teste. O conjunto de treino recebeu 2.160 registros, aproximadamente 70% da base limpa. Validação e teste receberam 463 registros cada, aproximadamente 15% por conjunto. A busca do Algoritmo Genético utilizou validação cruzada somente sobre os dados de treino. Após a seleção dos hiperparâmetros, treino e validação foram reunidos para o ajuste final, enquanto o conjunto de teste permaneceu isolado até a comparação definitiva entre os modelos.

6 Modelo de Referência

O modelo de referência é o Random Forest estabelecido na Fase 1. Esse algoritmo combina múltiplas árvores de decisão treinadas com amostras e subconjuntos de atributos, sendo apropriado para relações não lineares e para dados com variáveis de diferentes naturezas.

A configuração original foi reavaliada no mesmo conjunto de teste utilizado para o modelo otimizado. Essa escolha permite uma comparação direta, evitando que diferenças de particionamento sejam confundidas com ganhos decorrentes da otimização.

No teste final, o modelo de referência apresentou accuracy de 0,9914, precision de 0,9048, recall de 0,9048, F1-score de 0,9048 e AUC-ROC de 0,9942.

7 Otimização com Algoritmo Genético

O Algoritmo Genético foi implementado com a biblioteca DEAP. Cada indivíduo representa uma configuração possível do Random Forest. A população inicial é gerada aleatoriamente dentro de intervalos definidos previamente, e as gerações seguintes são produzidas por seleção, cruzamento, mutação e elitismo.

7.1 Representação dos genes

O cromossomo possui cinco genes:

Tabela 1: Hiperparâmetros codificados no cromossomo.

Gene	Tipo	Função
n_estimators	inteiro	Define a quantidade de árvores da floresta.
max_depth	inteiro	Limita a profundidade máxima de cada árvore.
min_samples_split	inteiro	Controla a quantidade mínima de amostras para uma divisão.
min_samples_leaf	inteiro	Define a quantidade mínima de amostras em uma folha.
max_features	categórico	Determina a estratégia de atributos avaliados nas divisões.

7.2 Operadores evolutivos

A seleção utiliza torneio, favorecendo indivíduos mais aptos sem eliminar completamente a diversidade da população. O cruzamento de dois pontos combina segmentos de dois cromossomos. A mutação uniforme altera genes de acordo com uma probabilidade de-

finida em cada experimento. O elitismo preserva os melhores indivíduos entre gerações, reduzindo a possibilidade de perda de soluções já encontradas.

7.3 Função de aptidão

A aptidão é representada pela tupla (**recall**, **F1-score**, **AUC-ROC**), calculada por validação cruzada estratificada com três partições. A comparação entre indivíduos é lexicográfica: primeiro é maximizado o recall médio da classe de hipotireoidismo; em caso de empate, o F1-score e a AUC-ROC são utilizados sucessivamente como critérios de desempate. O desvio-padrão do recall é registrado como informação de estabilidade. O recall foi priorizado porque um falso negativo corresponde a um registro positivo não identificado pelo modelo.

A avaliação é feita por meio do pipeline completo. Desse modo, imputação e codificação são ajustadas novamente dentro de cada partição da validação cruzada. Resultados de configurações já avaliadas são armazenados em cache para evitar treinamento repetido do mesmo conjunto de hiperparâmetros.

7.4 Protocolo experimental

Foram executados três experimentos independentes com diferentes populações, números de gerações, probabilidades e sementes. A diversidade das configurações permitiu observar a estabilidade da busca e reduziu a dependência de uma única execução aleatória.

Tabela 2: Resultados dos três experimentos do Algoritmo Genético.

Experimento	Recall médio	Desvio-padrão	AUC-ROC médio
Experimento 1	0,9596	0,0143	0,9958
Experimento 2	0,9697	0,0000	0,9959
Experimento 3	0,9596	0,0143	0,9956

O segundo experimento foi selecionado. O indivíduo vencedor utilizou 149 árvores, profundidade máxima igual a 20, mínimo de 8 amostras para divisão, mínimo de 5 amostras por folha e `max_features=log2`.

A Figura 1 apresenta o melhor recall encontrado em cada geração. O segundo experimento alcançou o maior valor desde a população inicial e o preservou ao longo da busca por meio do elitismo. Os experimentos 1 e 3 convergiram para valores semelhantes, mas inferiores ao resultado do experimento 2.

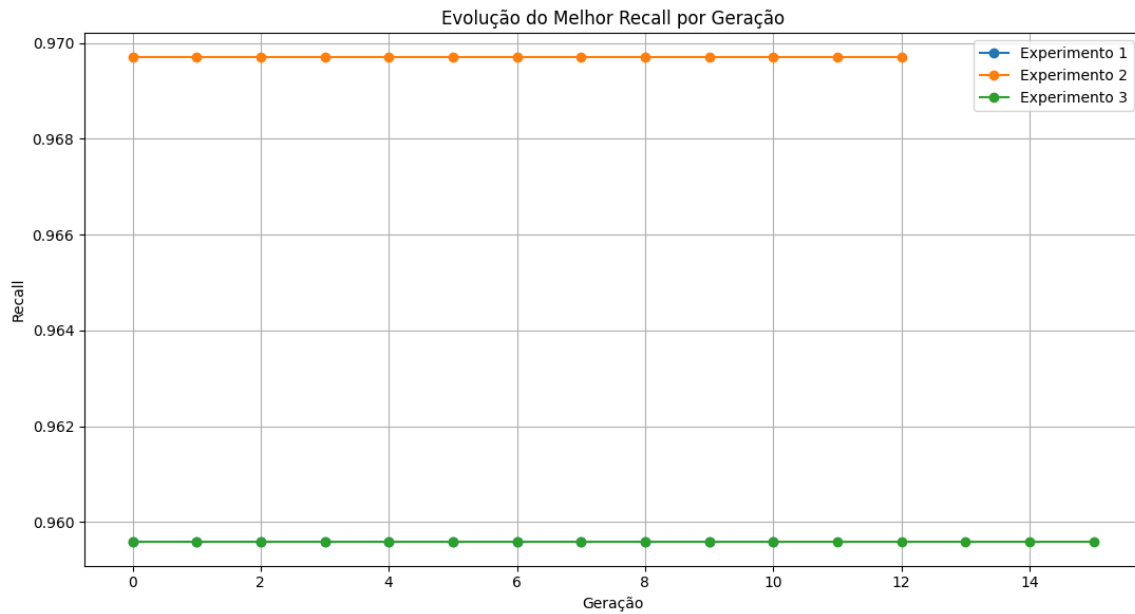


Figura 1: Evolução do melhor recall por geração nos três experimentos.

A evolução do recall médio da população, mostrada na Figura 2, permite observar o comportamento coletivo dos indivíduos. O crescimento mais consistente do experimento 2 indica que a população passou a concentrar configurações com melhor aptidão ao longo das gerações.

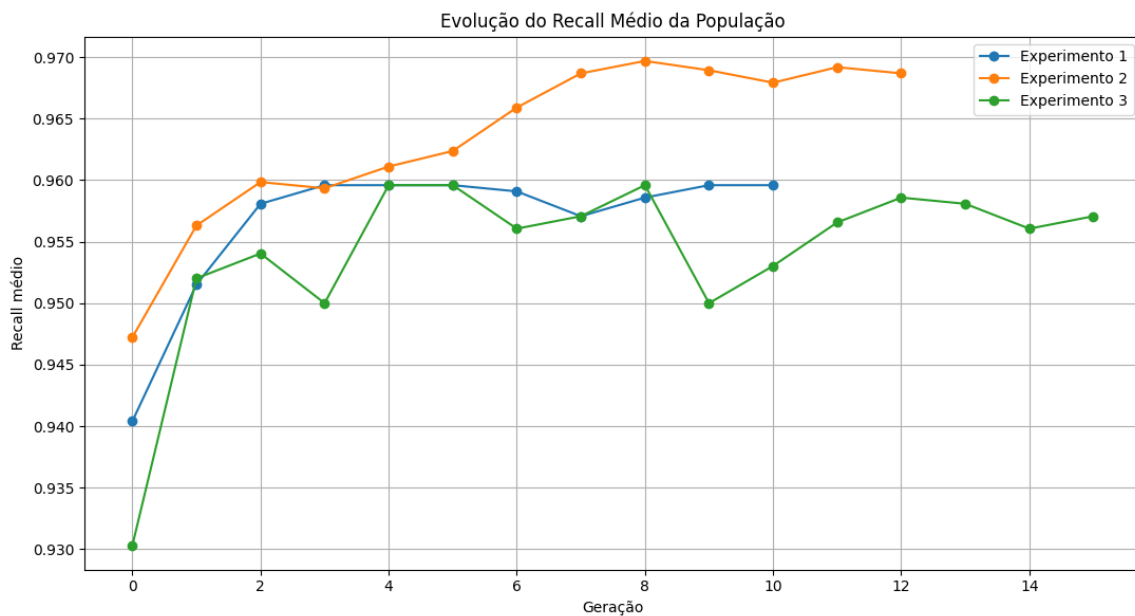


Figura 2: Evolução do recall médio da população em cada experimento.

8 Resultados e Comparação dos Modelos

Após a busca, o melhor indivíduo foi reconstruído no pipeline, treinado com os conjuntos de treino e validação reunidos e avaliado no conjunto de teste reservado.

Tabela 3: Comparação dos modelos no conjunto de teste.

Modelo	Accuracy	Precision	Recall	F1-score	AUC-ROC
Random Forest original	0,9914	0,9048	0,9048	0,9048	0,9942
Random Forest otimizado	0,9892	0,8333	0,9524	0,8889	0,9947

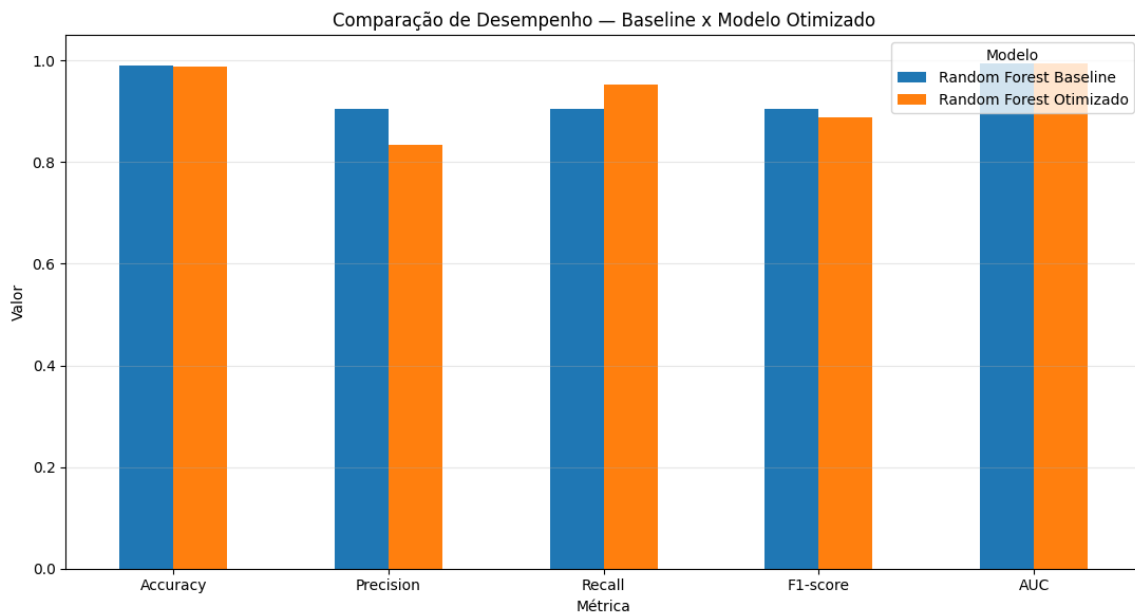


Figura 3: Comparação das métricas do Random Forest original e do modelo otimizado.

A otimização elevou o recall de 0,9048 para 0,9524, aumento de aproximadamente 4,76 pontos percentuais. Em contrapartida, a precision foi reduzida de 0,9048 para 0,8333 e o F1-score passou de 0,9048 para 0,8889. A accuracy apresentou pequena redução, enquanto a AUC-ROC permaneceu elevada e ligeiramente superior no modelo otimizado.

Os resultados não demonstram superioridade absoluta de um modelo sobre o outro. O modelo otimizado é mais sensível e reduz falsos negativos, mas produz mais falsos positivos. Em uma aplicação de triagem, esse comportamento pode ser aceitável quando a prioridade é encaminhar possíveis casos para revisão. A definição final do limiar de decisão dependeria de validação clínica, análise de custos e calibração das probabilidades.

As Figuras 4 e 5 detalham os erros dos dois modelos. O modelo original registrou dois falsos negativos e dois falsos positivos. O modelo otimizado reduziu os falsos negativos para um caso, mas aumentou os falsos positivos para quatro, comportamento coerente com o ganho de recall e a redução de precision.

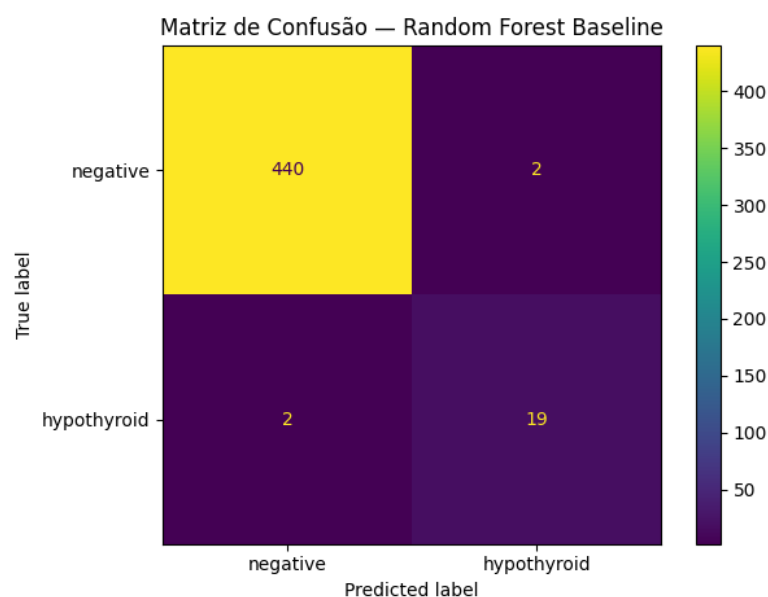


Figura 4: Matriz de confusão do Random Forest original.

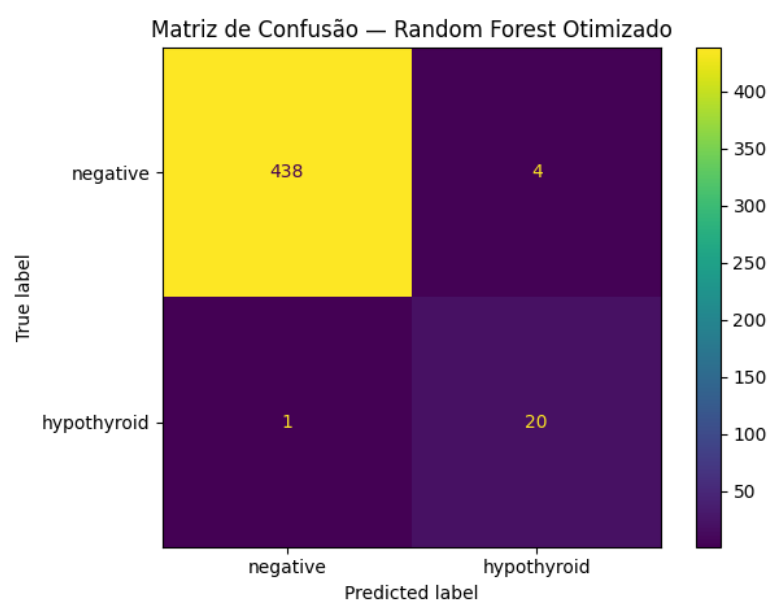


Figura 5: Matriz de confusão do Random Forest otimizado.

8.1 Importância global das variáveis

A análise de importância do Random Forest indicou maior contribuição global das variáveis laboratoriais TSH, FTI, TT4, T3 e T4U. TSH e FTI concentraram as maiores importâncias, com valores aproximados de 0,3142 e 0,3091, respectivamente.

Tabela 4: Principais variáveis segundo a importância global do modelo.

Variável	Importância
TSH	0,3142
FTI	0,3091
TT4	0,1804
T3	0,0718
T4U	0,0368

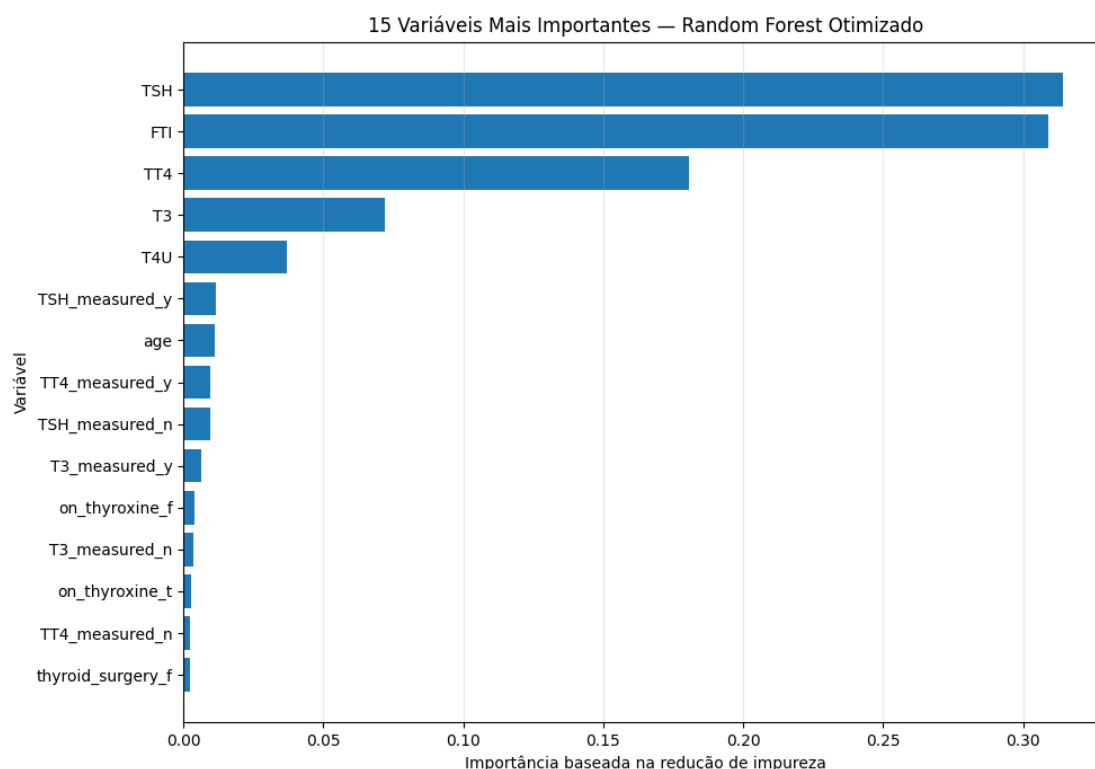


Figura 6: Quinze variáveis com maior importância global no Random Forest otimizado.

Essa análise descreve o comportamento global do modelo, mas não estabelece causalidade e não explica isoladamente uma predição individual. A importância também pode ser influenciada pela representação das variáveis e pelas correlações existentes nos dados.

9 Integração com Modelo de Linguagem

A etapa de linguagem natural utiliza uma LLM pré-treinada por meio da API da OpenAI. Seu papel é organizar os resultados do classificador em uma explicação clara e cautelosa. A LLM não realiza uma nova classificação e não altera a saída numérica do Random Forest.

9.1 Engenharia de prompt

O prompt foi construído com blocos separados para contexto, resultados, variáveis disponíveis, restrições e formato esperado. Entre as instruções utilizadas estão:

- reproduzir fielmente a classe e as probabilidades fornecidas;
- não converter a probabilidade em diagnóstico clínico;
- diferenciar importância global de explicação causal;
- informar quando um valor estiver ausente;
- não recomendar medicamento, tratamento ou conduta;
- utilizar linguagem objetiva e compreensível;
- incluir aviso de que o sistema possui finalidade acadêmica.

Uma entrada demonstrativa continha a classe prevista, as probabilidades das duas classes, os valores de TSH, FTI, TT4, T3 e T4U e a indicação de idade não informada. A chave de acesso da API é solicitada em tempo de execução e não é armazenada no notebook ou no repositório.

9.2 Avaliação da explicação

A qualidade da resposta foi verificada por seis regras implementadas no próprio notebook:

Tabela 5: Critérios de avaliação da resposta da LLM.

Critério	Verificação
Clareza e concisão	A resposta deve conter entre 50 e 300 palavras.
Fidelidade às probabilidades	As duas probabilidades enviadas no prompt devem aparecer na resposta.
Prudência clínica	O texto deve diferenciar previsão de diagnóstico e mencionar a necessidade de avaliação clínica.
Utilidade para apoio à decisão	A interpretação deve relacionar recall e falsos negativos.
Limitações apresentadas	Falsos positivos e falsos negativos devem ser mencionados.
Segurança	O texto não pode prescrever medicamento nem indicar início de tratamento.

A resposta registrada no notebook, composta por 152 palavras, foi aprovada nas seis verificações. Trata-se de uma avaliação automatizada e transparente da demonstração executada, não de uma validação clínica ou linguística abrangente. Uma análise mais completa exigiria outros perfis de entrada, revisão humana e testes de robustez.

10 API, Monitoramento e Autoscaling

O modelo otimizado foi disponibilizado por uma API desenvolvida com FastAPI. O artefato `random_forest_otimizado.joblib` contém o pipeline completo, incluindo pré-processamento e classificador, evitando divergências entre treinamento e inferência. Esse arquivo encontra-se versionado no diretório `models` da branch `Nirton`.

Tabela 6: Principais endpoints da API.

Endpoint	Método	Finalidade
<code>/health</code>	GET	Informa o estado do serviço e a disponibilidade do modelo.
<code>/predict</code>	POST	Recebe as 25 variáveis esperadas e retorna classe e probabilidades.
<code>/metrics</code>	GET	Expõe contadores e latência no formato Prometheus.
<code>/docs</code>	GET	Disponibiliza documentação interativa gerada pelo OpenAPI.

Os logs são emitidos em JSON e registram eventos operacionais sem incluir informações clínicas completas. O monitoramento utiliza contador de requisições, contador de erros e histograma de latência.

10.1 Implantação pública no Render

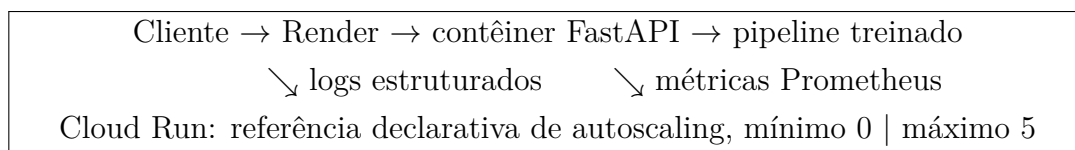
A API foi implantada no Render e encontra-se acessível em <https://tech-challenge-ia-saude.onrender.com>. A validação remota confirmou respostas HTTP 200 nos endpoints de saúde, predição, métricas e contrato OpenAPI. O endpoint `/health` informou o estado `ok` e confirmou a disponibilidade do arquivo `models/random_forest_otimizado.joblib`. O endpoint `/predict` recebeu uma observação real do conjunto de teste e retornou a classe prevista, as probabilidades das duas classes e o aviso de finalidade acadêmica.

O serviço utiliza o plano gratuito do Render. Consequentemente, o primeiro acesso após um período de inatividade pode apresentar latência adicional durante a inicialização. Essa característica é tratada no notebook por uma rotina de espera e novas tentativas. A API de predição utiliza exclusivamente o pipeline Random Forest treinado e não exige que o usuário forneça uma chave da OpenAI.

10.2 Configuração de autoscaling

A configuração do Google Cloud Run define escalonamento automático entre zero e cinco instâncias, alvo de quinze requisições concorrentes por instância, concorrência máxima igual a vinte, dois processadores virtuais e 2 GiB de memória. Esse arquivo representa uma

configuração declarativa de implantação e permanece como referência de uma arquitetura escalável. O projeto não apresenta evidência de execução dessa configuração no Cloud Run; a publicação efetivamente validada foi realizada no Render.



11 Testes e Reprodutibilidade

O repositório contém doze testes automatizados para carregamento do modelo, validação dos endpoints, tratamento de arquivo ausente, formato das probabilidades, erros de entrada e exposição das métricas. Todos os testes foram aprovados na validação final.

Além dos testes automatizados, foi realizada uma validação integrada com o artefato treinado na API pública do Render. Os endpoints `/health`, `/predict`, `/metrics` e `/openapi.json` retornaram status HTTP 200. A resposta de predição apresentou classe válida, probabilidades consistentes cuja soma é igual a um e o aviso de uso acadêmico.

O notebook foi estruturado para execução no Google Colab. Ele instala as dependências, clona diretamente a branch `Nirton`, prepara os dados, executa os experimentos, salva resultados, gera o modelo e valida os endpoints da API. As células registradas no arquivo foram executadas sem erros pendentes.

Como artefatos Jovlib dependem da versão da biblioteca usada na serialização, o `scikit-learn` foi fixado na versão 1.6.1, correspondente ao ambiente de treinamento. Essa decisão evita falhas de carregamento decorrentes de mudanças internas entre versões.

12 Desafios Enfrentados e Soluções Implementadas

Os pontos desta seção foram observados durante a implementação e os testes do projeto. O registro procura apresentar o que de fato exigiu revisão no código ou na organização da entrega, incluindo limitações que permaneceram após os ajustes.

12.1 Pré-processamento durante a validação cruzada

Ao organizar a busca de hiperparâmetros, foi necessário garantir que a imputação dos valores ausentes e a codificação das variáveis categóricas não fossem ajustadas antes da separação dos folds. Se isso ocorresse, cada fold teria acesso indireto a informações dos demais. Por esse motivo, o pré-processamento foi mantido dentro do Pipeline do `scikit-learn`. Em cada uma das três partições da validação cruzada, transformadores e classificador são ajustados novamente apenas com os dados disponíveis naquele treinamento.

12.2 Desbalanceamento da base e escolha do recall

Depois da remoção dos duplicados, a base ficou com 2.945 registros negativos e 141 registros de hipotireoidismo. Nesse cenário, uma accuracy elevada poderia coexistir com erros relevantes na classe minoritária. Por isso, o recall do hipotireoidismo foi colocado como primeiro critério da função de aptidão, seguido por F1-score e AUC-ROC nos empates. O resultado final tornou esse compromisso visível: o recall passou de 0,9048 para 0,9524, enquanto a precision caiu de 0,9048 para 0,8333. A matriz de confusão mostrou a redução de dois para um falso negativo e o aumento de dois para quatro falsos positivos.

12.3 Tempo de execução dos experimentos

O custo da busca ficou mais claro quando os três experimentos foram executados no Colab. Os tempos registrados foram 195,30 segundos, 591,72 segundos e 269,56 segundos. O segundo experimento, com população igual a 40, foi o mais demorado e também produziu o maior recall médio. Para evitar avaliações repetidas, foi utilizado um cache de fitness. Ao todo, 321 configurações foram treinadas e 462 resultados já calculados puderam ser reutilizados. Mesmo com esse recurso, uma nova execução completa continua levando vários minutos e depende da capacidade disponível no ambiente do Colab.

12.4 Execução no Colab e disponibilização da API

Na primeira execução integrada, o notebook utilizava o branch padrão do repositório, que ainda não continha todos os arquivos da API. O clone foi alterado para usar explicitamente a branch `Nirton`. Também foi necessário separar dois cenários: o modo `local`, que inicia o Uvicorn dentro do Colab, e o modo `remote`, que utiliza a implantação no Render. O acesso local depende da sessão do Colab e, portanto, é temporário. No Render, o endereço é público, mas o plano gratuito pode suspender o serviço por inatividade. A rotina de verificação passou a aguardar a inicialização e repetir o *health check* antes dos testes de predição.

12.5 Versão do scikit-learn e carregamento do Joblib

O arquivo Joblib precisa ser carregado com uma versão compatível da biblioteca utilizada no treinamento. Nos testes em um ambiente limpo, manter apenas uma faixa ampla de versões do `scikit-learn` não garantia essa compatibilidade. A versão foi fixada em 1.6.1, correspondente ao ambiente que gerou o modelo. Depois do ajuste, o pipeline foi carregado novamente e utilizado em uma predição de controle, tanto nos testes automatizados quanto na API.

12.6 Inclusão do modelo no repositório

O padrão `*.joblib` do arquivo `.gitignore` impedia inicialmente o versionamento do modelo. Como a API precisa desse arquivo para funcionar depois de uma clonagem limpa, foi adicionada uma exceção apenas para `models/random_forest_otimizado.joblib`. O artefato final possui 606.539 bytes e foi mantido no Git por ser pequeno o suficiente para o repositório e necessário para reproduzir a inferência sem repetir o treinamento.

12.7 Limites da interpretação gerada pela LLM

A integração com a LLM exigiu cuidado para não transformar uma previsão estatística em orientação médica. A chave da OpenAI é solicitada somente em tempo de execução e não fica gravada no notebook. O prompt informa as probabilidades, as métricas e os valores disponíveis, além de proibir diagnóstico e prescrição. A resposta demonstrativa foi verificada por seis regras implementadas no notebook e passou em todas elas. Essa verificação foi útil para confirmar a aderência ao prompt utilizado, mas permanece limitada a uma única entrada e não equivale a uma avaliação clínica da explicação.

13 Limitações e Considerações Éticas

Os resultados são experimentais e dependem da qualidade e representatividade da base utilizada. Não houve validação externa em outra instituição, estudo prospectivo, calibração clínica do limiar ou avaliação de impacto assistencial. Por esse motivo, as probabilidades não devem ser interpretadas como risco individual.

A aplicação não deve ser utilizada para diagnóstico, prescrição ou definição autônoma de conduta. Um eventual uso real exigiria revisão por profissionais de saúde, avaliação de vieses, governança dos dados, proteção de informações pessoais, rastreabilidade das versões, monitoramento de degradação e mecanismos de intervenção humana.

A explicação produzida pela LLM deve ser tratada como recurso de comunicação. Mesmo com instruções restritivas, respostas generativas podem apresentar variações ou incorreções. A saída numérica original e os registros da execução devem permanecer disponíveis para auditoria.

14 Conclusão

A Fase 2 ampliou a solução desenvolvida anteriormente ao incorporar otimização evolutiva, explicação em linguagem natural e uma camada de serviço observável. O Algoritmo Genético foi implementado com os operadores obrigatórios e avaliado em três experimentos independentes. A configuração selecionada aumentou o recall da classe de hipotireoidismo de 0,9048 para 0,9524 no conjunto de teste.

A comparação também demonstrou que o ganho de recall foi acompanhado por redução de precision e F1-score. O resultado reforça que a seleção de um modelo deve considerar o objetivo operacional e o custo dos diferentes tipos de erro, e não apenas uma métrica isolada.

A integração com a LLM apresentou explicação fiel e segura na avaliação registrada. A API foi implantada e validada publicamente no Render, enquanto o modo local permaneceu disponível para reprodução no Colab. O modelo treinado versionado, os testes automatizados, os logs, as métricas, o contêiner e a configuração declarativa de autoscaling completam a demonstração técnica. Como trabalhos futuros, recomenda-se ampliar a avaliação das explicações, calibrar probabilidades, testar diferentes limiares, validar o modelo em uma base externa e realizar implantação controlada com monitoramento contínuo e avaliação de escalabilidade.

15 Referências

- UCI Machine Learning Repository. *Thyroid Disease Dataset*. Disponível em: <https://archive.ics.uci.edu/dataset/102/thyroid+disease>
- DEAP Project. *Distributed Evolutionary Algorithms in Python*. Disponível em: <https://deap.readthedocs.io/>
- Scikit-learn. *User Guide*. Disponível em: <https://scikit-learn.org/>
- FastAPI. *Documentation*. Disponível em: <https://fastapi.tiangolo.com/>
- Prometheus. *Documentation*. Disponível em: <https://prometheus.io/docs/>
- Google Cloud. *Cloud Run Documentation*. Disponível em: <https://cloud.google.com/run/docs>
- Render. *Web Services Documentation*. Disponível em: <https://render.com/docs/web-services>
- OpenAI. *API Documentation*. Disponível em: <https://platform.openai.com/docs/>
- BREIMAN, L. Random Forests. *Machine Learning*, v. 45, p. 5–32, 2001.
- HOLLAND, J. H. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.